

Building Web CAD from Scratch

B-Rep, Native Documents, and STEP Interoperability

Architecture exploration note

Core position

A browser-based CAD system should separate exact B-Rep authoring from open-format interchange. The native document is responsible for continued editing and state recovery; STEP is responsible for import and export across CAD systems. They should not be forced into the same file format.

The proposed direction is to run an exact geometry kernel in the browser (for example, OCCT compiled to WebAssembly), use B-Rep as the geometric source of truth, keep modeling and document semantics in a separate application model, and invoke a translation layer only when users import or export STEP.

Executive summary

Question	Recommendation
Canonical geometry in the browser	Keep exact B-Rep inside the WASM/kernel layer; JavaScript holds handles and IDs rather than materialized topology.
Native document	Do not equate it with STEP. Persist feature/assembly/metadata plus a B-Rep checkpoint and an optional preview.
B-Rep persistence	For an OCCT-based prototype, use OCCT native B-Rep serialization first; evolve to a versioned binary payload later.
STEP	Treat it as an interchange boundary: import into canonical B-Rep, export from the current canonical B-Rep.
Complex STEP data	Plan for XDE/XCAF, not only raw TopoDS_Shape, if assemblies, names, colors, units, or product structure matter.
Performance	Avoid repeated B-Rep copies between the JS heap and WASM memory. Tessellated meshes are for rendering, not geometry truth.

1. Goals and non-goals

The goal is a genuine Web solid modeler: users can create and modify exact geometry directly in the browser and can import or export STEP when interoperability is needed. Rendering is not the primary architectural challenge; the critical boundaries are the geometry kernel, document semantics, persistence, and translation.

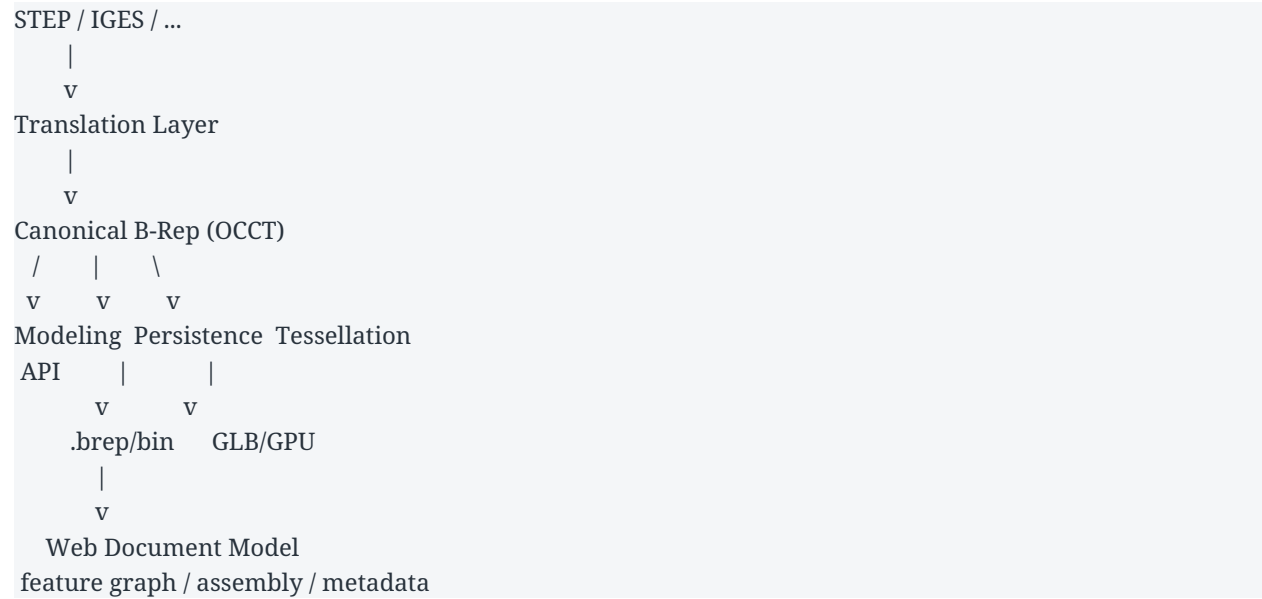
A useful minimum capability loop is:

- Create primitives and/or sketch-derived solids
- Perform B-Rep operations such as Boolean, Extrude, and Fillet
- Tessellate quickly into a WebGL/WebGPU viewport
- Save and reopen a native Web CAD document
- Import STEP and continue editing in the browser
- Export the current model as STEP, GLB, or STL

Non-goals

The first release does not need to reconstruct the original parametric history of an imported STEP file, nor does it need Fusion- or Onshape-class parametric capability from day one. Imported STEP geometry can enter the system as an ImportedBody/base feature, with new Web-native operations layered on top.

2. Overall architecture



The key principle is that translation is a boundary concern, not a concept that leaks through the application. Whether geometry is created natively in the browser or imported from STEP, it should become the same canonical B-Rep once it enters the system.

3. Why the native document should not be STEP

STEP is valuable because it is neutral, standardized, and broadly interoperable across CAD kernels. But it is not naturally equivalent to a Web CAD authoring document. A native document also needs to preserve parameters, feature graphs, assembly instances, selection semantics, application metadata, versioning, caches, and other authoring state.

Native document	Exchange
Feature graph	STEP
Parameters	AP2xx schema
Assembly semantics	<----> CAD interchange
B-Rep checkpoint	
Topology identity / references	
Preview / tessellation cache	

A robust principle is therefore: the document format is a container; B-Rep serialization is one payload within it; STEP is an external interchange representation.

4. How to persist B-Rep

If the geometry kernel is OCCT, the simplest prototype choice is OCCT native B-Rep serialization. It can restore a TopoDS_Shape together with underlying curves, surfaces, and topology without translating through STEP on every save and reopen cycle.

Format	Exact B-Rep	Internal persistence	Interoperability	Recommended role
OCCT .brep	Yes	High	Low	OCCT-native snapshot
STEP	Yes	Medium	High	External interchange
Parasolid x_t/x_b	Yes	High	High	Native to a Parasolid stack
GLB	No - mesh	Low	High	Preview / realtime rendering

Recommended container structure

```

model.webcad
manifest.json    # schemaVersion, units, kernelVersion
features.json/bin # feature graph / parameters
geometry.brep/bin # exact B-Rep checkpoint
topology.bin     # optional persistent identity mapping
preview.glb      # optional fast preview

```

This keeps the document format independent from a particular kernel serialization. A future OCCT upgrade, a new B-Rep encoding, or richer assembly semantics can be introduced without replacing the entire file format.

5. STEP import and export

STEP import should be treated as an explicit translation transaction. After import, the application should work with canonical bodies and parts rather than with STEP entities directly.

```

.step bytes
-> STEP reader
-> transfer / unit handling
-> healing / validation
-> TopoDS_Shape or XCAF document
-> register geometry handles
-> tessellate
-> GPU viewport

```

Export runs in the opposite direction: serialize the current canonical B-Rep into a STEP representation and return the result through a browser Blob, File System Access API, or equivalent download path. If the user performs Boolean, Fillet, Transform, or other operations in the browser, the exported STEP should contain that modified exact geometry.

```
STEP import -> canonical B-Rep -> Web edits -> canonical B-Rep -> STEP export
```

6. Editing after STEP import

Do not assume that STEP can be reconstructed into the original sketch/extrude/fillet feature history from the source CAD system. In most cases STEP gives you result geometry and product structure, not the original authoring graph.

```
Document
  ImportedBody_01    # source = STEP
    exact B-Rep
  Feature_01        # Boolean Cut added in Web
  Feature_02        # Fillet added in Web
  CurrentBody       # current canonical B-Rep
```

This produces a direct-modeling workflow: the imported body is a stable base feature, and subsequent Web operations are applied on top of it. Feature recognition can be added later as a separate capability rather than becoming a prerequisite for STEP import.

7. The hard problem: topology references and recompute

For non-trivial B-Rep models, the hard part is usually not file size. It is maintaining stable references from downstream features to faces and edges as upstream operations change topology. A Boolean or dimensional edit may invalidate a simple reference such as Face_17.

```
Bad:
  Fillet -> Face_17

Better:
  Fillet -> semantic / persistent reference
  e.g. "top planar face produced by Extrude_03"
```

In the medium term, the system will need persistent naming and/or topology identity mapping. A first version can reduce this risk by limiting recomputation scope, treating imported geometry as an immutable base, or favoring direct edits before full parametric dependency tracking is introduced.

8. WebAssembly boundary

Exact B-Rep data should remain in WASM linear memory or the kernel heap for as long as possible. The JavaScript/TypeScript layer should invoke modeling operations through handles rather than repeatedly materializing topology and surface structures as JS objects.

```
JavaScript / TypeScript
  BodyId / FaceId / OperationId
  |
  | API boundary
  v
```

```
WASM / OCCT
TopoDS_Shape
BRepAlgoAPI_*
BRepMesh_*
STEP / XCAF translators
```

Application APIs should look like `kernel.booleanCut({ target, tool })` instead of copying a complete B-Rep representation into JavaScript. Tessellation output - vertex/index buffers and attributes - is the data that should cross into the rendering pipeline.

9. TopoDS_Shape or XCAF/XDE?

For a single-part prototype, `TopoDS_Shape` is sufficient. If realistic STEP assemblies are in scope, it is worth designing for XDE/XCAF early. Real STEP data may carry assembly hierarchy, part names, colors, layers, units, and product structure in addition to geometry.

```
Web CAD Document
Product / Scene Tree
  Assembly
    Instance -> Part
Metadata
  name / color / units / layer
Geometry
  handle -> TopoDS_Shape
```

Recommendation: if assembly STEP is a real target, separate the CAD document model from the shape model in the first architecture. Otherwise the migration from a shape-only system to a product-structure system will be disproportionately disruptive.

10. Recommended MVP

A useful first version does not need a broad modeling command set. The objective is to validate the full lifecycle and expose the architectural bottlenecks.

Stage	Capability	Primary validation
MVP-0	Box/Cylinder + Boolean + viewport	Does the OCCT/WASM-to-rendering boundary work well?
MVP-1	Save native document + .brep checkpoint	What is the recovery cost after browser restart?
MVP-2	STEP import + edit + STEP export	What are the round-trip and geometry-quality characteristics?
MVP-3	Assembly/name/color through XCAF	Can realistic STEP document semantics be preserved?
MVP-4	Persistent references + incremental recompute	How stable are parametric edits as topology changes?

11. Current architectural decisions

- Geometry kernel: OCCT compiled to WebAssembly, or an equivalent exact B-Rep kernel.
- Canonical geometry: kernel-native B-Rep; mesh is never the geometric source of truth.
- Native document: versioned container holding feature/assembly/metadata plus a B-Rep checkpoint.
- B-Rep persistence: use OCCT .brep/native serialization for the prototype; evolve to a versioned binary payload later.
- Interchange: STEP is the first-priority CAD format; GLB/glTF serves visualization and realtime ecosystems; STL serves simple mesh interchange.
- Imported STEP: represent it as an ImportedBody/base feature rather than inventing parametric history.
- Assembly: if in scope, prefer XCAF/XDE over a pure TopoDS_Shape model.
- Web boundary: keep B-Rep resident in WASM; JavaScript holds handles, document semantics, and render buffers.

12. Open questions

- What are real serialize/deserialize throughput and peak-memory costs for complex B-Rep models in the browser?
- Is IndexedDB or OPFS a better fit for large B-Rep checkpoints and incremental saves?

- What are the practical OCCT/WASM STEP/XCAF translation limits for 10 MB, 100 MB, and 500 MB models?
- Should the modeling kernel run entirely inside a Worker, potentially using shared memory?
- Should persistent naming come from kernel capabilities, application-level semantic references, or a hybrid?
- How should native-document migrations be designed so files are not tied to a specific OCCT build?
- How should assembly instances and shared geometry be represented without duplicating B-Rep data?

Next experiment worth running

Build an end-to-end benchmark with a medium-complexity STEP file: import it in the browser, translate to OCCT B-Rep, tessellate it, persist a native checkpoint, close and restore the document, perform one Boolean or Fillet, and export STEP again. Capture wall-clock time, WASM memory, file sizes, and topology/geometry differences. This experiment will reveal architectural bottlenecks faster than further debate over format names.

Conclusion

Translation is unavoidable, but it should be constrained to system boundaries. A Web CAD system should own its canonical B-Rep and document semantics internally; when interoperability is required, it translates the current state to STEP or translates STEP into the canonical model. This preserves efficient continued editing, an evolvable native document, and open interoperability with the broader CAD ecosystem.